

Chapter 6

Setting Up to Learn

The computer you need, installing and checking the book's tools, companion files, documentation, learning with AI assistants, and a study plan you can keep.

[Draft chapter](#)

17 September 2026

Versions and install steps checked against official sources

Contents

Chapter 6. Setting Up to Learn

Why this matters

In plain English

6.1 The computer you need (and don't)

6.2 Your toolkit at a glance

6.3 Installing, in order

6.4 The companion files and a folder that stays tidy

6.5 Keyboard, files, and the terminal: the basics that save hours

6.6 Reading documentation

6.7 Learning with AI assistants without letting them think for you

6.8 Building a study habit

6.9 How to use the exercises and answers

Common mistakes and how to spot them

In the real world: Meera sets up

Tools

The project: set up, check, and plan your first 90 days

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 6. Setting Up to Learn

Part 0 — First Principles: Data from Zero

Chapter at a glance

You will learn to: check whether your computer is ready, and what to do if it isn't · install the book's core tools in a sensible order, and confirm each one works · organize the companion files so you can always find them · use the keyboard, files, and terminal basics that save hours · read official documentation to settle a question for yourself · use AI assistants to learn faster without letting them do your thinking · build a study plan and a weekly habit you can keep · get the most from the exercises and answers.

Before you start: Chapters 1–5. Chapter 2 (files, folders, extensions, and paths) is especially useful here.

Time needed: 3–4 hours, most of it installing. Spread the installs over two or three sittings if you like.

Tools: the computer you'll study on, an internet connection, and permission to install software on it.

Practice data: the Riverstone mini database (`riverstone_setup.sql`) and a small script, `check_setup.py` , both in the companion files (Appendix E). Every version, install method, and output in this chapter was checked against official sources or run, in September 2026.

Why this matters

From Chapter 10 onward, every chapter asks you to type, run, break, and fix things: formulas, queries, scripts, dashboards. Readers who set up properly before they need to get through those chapters in a fraction of the time, because a missing driver or a wrong Python version never interrupts them halfway through an exercise. Readers who skip setup often lose an evening to an installation problem and quietly stop.

This chapter also covers the other half of setting up: **you**. A plan you can keep, a weekly rhythm, a way to use AI assistants that helps you learn instead of replacing the learning, and the habit of checking the official documentation when two sources disagree. It all decides whether you finish.

In plain English

Think of a kitchen before cooking a big meal. A good cook doesn't start chopping and then discover there's no gas in the cylinder. They check the stove, lay out the knives and pans, read the recipe once through, and put the spices within reach. Cooks call it *mise en place*: everything in its place.

Setting up to learn is the same:

- **The stove and utensils** are your computer and the tools: a spreadsheet, a database, Python, Power BI.
- **The pantry** is the companion files: the Riverstone data you'll practice on.
- **The recipe book** is this book, plus the official documentation for each tool, which is where you look when the recipe is unclear.
- **The meal plan** is your study plan: what you'll cook this week, and when.

You don't need the most expensive equipment, only the right equipment, working, and in reach.

6.1 The computer you need (and don't)

You don't need a new or high-end computer to become a data analyst. Almost everything in Parts 0 to II runs comfortably on an ordinary laptop from the last five or six years.

	Enough to start	Comfortable
Operating system	Windows 10 or 11, or a recent macOS	Windows 11 or the latest macOS your Mac supports
Memory (RAM)	8 GB	16 GB
Free disk space	20 GB	50 GB or more
Screen	a laptop screen	a second monitor, so the book or documentation sits next to your work
Internet	enough to download installers	enough for video calls and cloud tools later

Three situations need a little planning:

- **A Mac.** Everything in the book runs on a Mac except **Power BI Desktop**, which is Windows only. The usual answer in Microsoft's Q&A forums is to run Windows 11 in a virtual machine (for example, Parallels Desktop). The browser version of Power BI can view and interact with reports but can't build the data models Chapter 16 teaches, and it needs a work or school account. You can do everything else on your Mac and handle Chapter 16 separately.
- **A Chromebook, tablet, or phone.** These can run Google Sheets, Excel for the web, and online SQL playgrounds, which is enough for Chapters 10, 11, and much of 12 and 13. They can't run Python properly, the databases, or Power BI Desktop. Plan to borrow or buy a laptop before Chapter 17.
- **A work laptop you can't install software on.** Many companies lock their laptops, for good security reasons. Ask your IT team; learning tools like PostgreSQL, Python, and DBeaver are commonly approved. Some tools install without administrator rights: Microsoft's documentation notes that Power BI Desktop from the Microsoft Store doesn't need admin rights, and Python's install manager installs for your own user. The story later in this chapter shows one way through.

Watch out: Windows 10. Microsoft stopped regular security updates for Windows 10 in October 2025. It still runs every tool in this book, but a computer that no longer gets security updates is a risk when you start connecting to company data. If you can upgrade to Windows 11, do.

6.2 Your toolkit at a glance

The core toolkit, in the order the book first uses it



Later parts install their own tools when you reach them: dbt, the command line, cloud accounts, containers, and ML libraries.

Figure 6.1 – Five groups of tools cover the whole analyst path. Later parts add their own tools when you reach them.

Tool	What it's for	First used	Cost	Runs on	Version at the time of writing
Excel or Google Sheets	spreadsheets	Chapter 10	Excel for the web and Google Sheets are free with a Microsoft or Google account; desktop Excel needs a Microsoft 365 subscription	Windows, Mac, browser	always updated
PostgreSQL	the book's main database	Chapter 12	free	Windows, Mac, Linux	18 (the current major version, supported until November 2030)
MySQL Community Server	the second database	Chapter 12	free	Windows, Mac, Linux	9.7 LTS (8.4 LTS is also fine)
DBeaver Community	the app you type SQL into	Chapter 12	free	Windows, Mac, Linux	26.2
Power BI Desktop	dashboards and data models	Chapter 16	free (Microsoft describes the desktop app as free)	Windows only	updated monthly
Python	programming for data	Chapter 17	free	Windows, Mac, Linux	3.13 or 3.14
VS Code	the editor for Python, SQL files, and notes	Chapter 17	free	Windows, Mac, Linux	updated monthly
Git	saving versions of your work	Chapter 26	free	Windows, Mac, Linux	2.55

A few words on these choices. **Free tools only**, so nobody is locked out. **PostgreSQL and MySQL both**, because between them they cover most databases an analyst meets (Chapter 12 explains). **DBeaver**, because one app works with both databases. **Power BI**, because it's widely used and free to start with; Chapter 16 also translates the ideas to Tableau and Looker. **Python with VS Code**, because it's free, runs everywhere, and the same editor serves you from your first script to production code in Part V.

Simplification note: versions move. Tools release new versions every few months. The book's examples were tested on PostgreSQL 16, MySQL 8.0, and Python 3.11 and 3.13, and use features that work the same way in the newer versions listed above. If a newer version than the table shows is available when you read this, install it. Appendix B keeps current instructions.

6.3 Installing, in order

Install in this order. Each step ends with a check, so a problem shows up immediately, not three chapters later. Use the official website for every tool; download sites that repackage installers sometimes bundle unwanted software.

Step 1. A spreadsheet

If you already have Excel through work or a Microsoft 365 subscription, you're done. If not, either works:

- **Google Sheets:** sign in to a Google account and open Google Sheets in your browser.
- **Excel for the web:** sign in with a free Microsoft account at Microsoft 365 on the web. Files are saved in OneDrive, which comes with 5 GB of free storage.

Check: create a sheet, type `2.5` in A1 and `=ROUND(A1,0)` in B1. You should see `3`. (Section 6.6 explains why that's worth checking.)

Tool note. Microsoft ran a free *desktop* Excel editing preview in some markets until July 2026. It has ended: the free desktop apps now open and view files, and editing is free only in the web versions. Chapters 10 and 11 point out the few features that need desktop Excel.

Step 2. The databases and DBeaver

Chapter 12, section 12.3, has full, tested steps for both databases and DBeaver on Windows, macOS, and Linux. You don't need to repeat them here; do them now, and come back. A summary:

1. **Install PostgreSQL** from the official PostgreSQL website. On Windows, the page recommends the interactive installer by EDB, which also includes pgAdmin and StackBuilder (you won't need either). On a Mac, the same installer works, or Postgres.app, a small app that runs PostgreSQL from the menu bar. Choose the current version, and **write down the password** you set for the `postgres` user.
2. **Install DBeaver Community.** It includes its own Java, so there's nothing else to install.

3. **Install MySQL** (optional until Chapter 12's MySQL sections). Choose the current LTS release, and write down the `root` password.
4. **Create the database and load the data**, as in section 12.3: `CREATE DATABASE riverstone;` in PostgreSQL, then run `riverstone_setup.sql`; in MySQL, run `riverstone_setup_mysql.sql`, which creates the database for you.

Check: in DBeaver, connected to `riverstone`, run the same query in each database.

```
SELECT COUNT(*) FROM order_items;
```

```
count
-----
      19
(1 row)
```

```
SELECT COUNT(*) FROM order_items;
```

```
+-----+
| COUNT(*) |
+-----+
|        19 |
+-----+
```

Nineteen order lines: the same twelve orders you followed through Chapters 1 to 5. If you see an error instead, the troubleshooting notes in section 12.3 cover the common ones, including MySQL's "Public Key Retrieval is not allowed".

Step 3. Python and the analyst's packages

Install Python from python.org:

- **Windows:** python.org now recommends the **Python install manager**, available from the python.org downloads page or the Microsoft Store (they're the same tool). The older full installer is being retired. After installing, open a terminal (section 6.5) and run `py install 3.14` if it didn't install a version for you. The commands `python` and `py` then start Python.
- **Mac:** download the macOS installer package from python.org and run it. When it finishes, open the new *Python 3.14* folder in *Applications* and double-click **Install Certificates.command**, which lets Python download packages securely. On a Mac, the command is `python3`.
- **Linux:** use your distribution's packages or python.org's source; most distributions already include a recent Python 3.

Create a folder and a virtual environment for the book. A **virtual environment** is a private copy of Python's package list for one project, so installing something for this book can't break anything else on your computer. Chapter 17 explains it properly; for now, follow the steps. In a terminal, go to your book folder (section 6.4) and run:

```
# Windows (PowerShell or Command Prompt)
python -m venv .venv
.venv\Scripts\activate
python -m pip install pandas openpyxl matplotlib jupyterlab

# macOS or Linux (Terminal)
python3 -m venv .venv
source .venv/bin/activate
python -m pip install pandas openpyxl matplotlib jupyterlab
```

When the environment is active, your terminal prompt starts with `(.venv)`. Each time you open a new terminal to work on the book, run the `activate` line again. If PowerShell refuses to run the activation script, the Python documentation's fix is to run `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser` once, or to use Command Prompt instead.

Check: run the companion script `check_setup.py` from the `ch06` folder:

```
python check_setup.py
```

On a correctly set-up computer, it prints:

```
python      3.13.13  OK
pandas      3.0.5    OK
openpyxl    3.1.5    OK
matplotlib  3.11.2   OK
jupyterlab  4.6.3    OK
All set.
```

Your version numbers may be newer; that's fine. If something is missing, the script says exactly what to run. Here's a real run on a computer where only one package had been installed:

```
python      3.11.15  OK
pandas      -        MISSING: run  python -m pip install pandas
openpyxl    3.1.5    OK
matplotlib  -        MISSING: run  python -m pip install matplotlib
jupyterlab  -        MISSING: run  python -m pip install jupyterlab
Fix the lines above, then run this script again.
```

Watch out: "pip installed it, but Python can't find it." This almost always means the package went into a different Python from the one running your script, often because the virtual environment wasn't active. Activate `.venv`, then install with `python -m pip install ...` rather than plain `pip install ...`, so the package goes into the Python you're actually using.

Step 4. VS Code

Install VS Code from its official website. Open it, go to the *Extensions* view, and install two extensions published by Microsoft: **Python** and **Jupyter**. Then use *File* → *Open Folder* to open your book folder.

Check: press `Ctrl+Shift+P` (`Cmd+Shift+P` on a Mac), type *Python: Select Interpreter*, and choose the one inside `.venv`. Create a file called `hello.py` containing `print("ready")`, and run it with the play button. The terminal at the bottom should print `ready`.

Step 5. Git

- **Windows:** install **Git for Windows** from the official Git website, accepting the default options.
- **Mac:** the Git website lists three ways; the simplest is to run `xcode-select --install` in Terminal, which installs Apple's command line tools, including Git. (Homebrew users can run `brew install git`.)
- **Linux:** install the `git` package from your distribution.

Check: in a terminal, run `git --version`. You'll see a version number such as `git version 2.55.0`. Chapter 26 teaches how to use it.

Step 6. Power BI Desktop (Windows)

On Windows, install **Power BI Desktop** from the **Microsoft Store**. Microsoft recommends the Store version because it updates itself and doesn't need administrator rights. Microsoft lists at least 2 GB of free memory (4 GB or more recommended) and a screen of at least 1440×900. Signing in matters later, when you publish and share reports through the Power BI service (Chapter 16).

On a Mac, skip this step for now, and decide before Chapter 16 whether to use a Windows virtual machine, a Windows computer at work or college, or to follow Chapter 16 by reading and doing its exercises on a borrowed machine.

Check: open Power BI Desktop, close the welcome screen, and you should see an empty report canvas.

6.4 The companion files and a folder that stays tidy

The **companion files** (Appendix E) hold every dataset, script, and workbook the book uses. So far that includes:

Folder or file	What it is	Used in
ch02/	four orders as CSV, Excel, JSON, XML, and Parquet; <code>api_demo.py</code>	Chapter 2
riverstone_setup.sql and mysql/riverstone_setup_mysql.sql	the mini database (first quarter of 2026)	Chapters 12, 13, and many examples
riverstone_2025_setup.sql and its MySQL twin	the one-year database (2025)	Chapter 13 onward
postgresql/ch12_lab_postgresql.sql , mysql/ch12_lab_mysql.sql , new_suppliers.csv	the build-a-database lab	Chapter 12
mysql/ch12_queries_mysql.sql , mysql/ch13_queries_mysql.sql	every query in Chapters 12 and 13, tested in MySQL	Chapters 12, 13
ch04/numbers_practice.xlsx	the numbers workbook	Chapter 4
ch06/check_setup.py	the setup checker	this chapter

Later chapters add their own folders. Appendix E gives the address to download them from and lists every file.

Set up one home for everything, and keep it that way:

```
analyst-to-architect/
  companion/           the downloaded companion files, unchanged
  work/
    ch10/              your own files for each chapter
    ch12/
    ...
  notes/              your study log, plans, and questions
  .venv/              the Python environment from section 6.3
```

Two rules keep it useful. **Never edit the companion files themselves:** copy what you need into `work/` first, so you can always start again from a clean copy. And **name your files the way Chapter 2 (section 2.4) taught:** dates as year-month-day, lower-case words joined with hyphens or underscores, and no "final".

6.5 Keyboard, files, and the terminal: the basics that save hours

Keyboard shortcuts worth learning this week

Action	Windows	Mac
Copy, cut, paste	Ctrl+C , Ctrl+X , Ctrl+V	Cmd+C , Cmd+X , Cmd+V
Undo, redo	Ctrl+Z , Ctrl+Y	Cmd+Z , Cmd+Shift+Z
Find in a page or file	Ctrl+F	Cmd+F
Save	Ctrl+S	Cmd+S
Switch between apps	Alt+Tab	Cmd+Tab
Open the search for apps and files	Windows key, then type	Cmd+Space , then type
Take a screenshot of part of the screen	Windows+Shift+S	Cmd+Shift+4
Command palette in VS Code	Ctrl+Shift+P	Cmd+Shift+P

Chapters 10, 12, and 17 add the shortcuts for spreadsheets, DBeaver, and VS Code. Learn a few at a time: use one new shortcut deliberately for a week, until your hands do it on their own.

Files

From Chapter 2, make sure you can: **show file extensions** (so `orders.csv` doesn't appear as plain `orders`); **find a file's full path** (Windows: hold `Shift` , right-click the file, *Copy as path*; Mac: select the file, then `Option+Cmd+C`); and **unzip** a download before opening what's inside it, because opening files directly from inside a zip is a common cause of "my changes disappeared".

The terminal

A **terminal** is a window where you type commands instead of clicking. You'll use it for Python, Git, and later for much more (Chapter 34 teaches it properly). For now, four things are enough:

To do this	Type	Example
Open a terminal	Windows: search for <i>Terminal</i> or <i>PowerShell</i> . Mac: search for <i>Terminal</i> . In VS Code: <i>Terminal</i> → <i>New Terminal</i> (opens in your current folder).	
See where you are	<code>pwd</code> (PowerShell, Mac, Linux); <code>cd</code> on its own in Command Prompt	
Move into a folder	<code>cd</code> followed by the folder name	<code>cd analyst-to-architect</code>
Run a Python script	<code>python</code> (Mac: <code>python3</code>) followed by the file name	<code>python check_setup.py</code>

The most common beginner error is running a command in the wrong folder, which gives "No such file or directory" or "can't open file". When you see that, check where you are first.

6.6 Reading documentation

Every tool in this book has **official documentation**: the manual written by the people who make it. When a tutorial, a colleague, and an AI assistant disagree, the documentation settles it.

A real question: how does 2.5 round?

Chapter 4 said to round at the end, not in the middle. Here's a follow-up question that sounds trivial: *what does rounding 2.5 to a whole number give?* The same question, in three of the book's tools.

In PostgreSQL:

```
SELECT ROUND(2.5)           AS exact_half,
       ROUND(3.5)           AS exact_three_half,
       ROUND(2.5::double precision) AS float_half,
       ROUND(3.5::double precision) AS float_three_half;
```

exact_half	exact_three_half	float_half	float_three_half
3	4	2	4

(1 row)

In MySQL:

```
SELECT ROUND(2.5) AS exact_half,
       ROUND(3.5) AS exact_three_half,
       ROUND(2.5E0) AS float_half,
       ROUND(3.5E0) AS float_three_half;
```

exact_half	exact_three_half	float_half	float_three_half
3	4	2	4

In Python:

```
print(round(2.5), round(3.5), round(2.675, 2))
```

```
2 4 2.67
```

And in a spreadsheet, `=ROUND(2.5,0)` gives 3 and `=ROUND(2.675,2)` gives 2.68 .

So 2.5 rounds to 3 in some places and to 2 in others, and 2.675 rounds to 2.68 in a spreadsheet but 2.67 in Python. Who's right? This is exactly the kind of question to settle in the documentation, not by guessing:

- **The PostgreSQL manual** (*Mathematical Functions and Operators*, `round`): "For *numeric* , ties are broken by rounding away from zero. For *double precision* , the tie-breaking behavior is platform dependent, but 'round to nearest even' is the most common rule." 2.5 typed on its own is *numeric* , so it goes to 3; converted to *double precision* , it goes to the even number, 2.
- **The MySQL manual** (`ROUND()`): exact-value numbers round "away from zero"; for approximate-value numbers, "the result depends on the C library", and on many systems that means "round to nearest even". 2.5E0 is written in scientific notation, which makes it an approximate value.
- **The Python documentation** (built-in `round`): "if two multiples are equally close, rounding is done toward the even choice". And a note: " `round(2.675, 2)` gives 2.67 instead of the expected 2.68 . This is not a bug": 2.675 can't be stored exactly as a floating-point number (Chapter 2, section 2.1), and the stored value is a hair below 2.675.

Rounding halves to the nearest even number is called **banker's rounding**, or round-half-to-even. It's used because, over many values, rounding every half up nudges totals upward, while rounding to even balances out. Neither rule is wrong. What's wrong is not knowing which one your tool uses, and then wondering why a Python report and an Excel report differ by a rupee.

How to read a documentation page

You don't read documentation front to back. You go in with a question and look for five things:

1. **The signature**: the name and what goes in, such as `ROUND(number, num_digits)` .
2. **The description**: what it returns, in one or two sentences. Read every word; the details (like "for *numeric* ") are usually the point.
3. **Examples**: run one yourself, exactly as written, before changing it.
4. **Notes and warnings**: often in a box. This is where `round(2.675, 2)` was explained.
5. **The version**: documentation sites usually cover several versions. Make sure you're reading the one you installed (for example, PostgreSQL's pages have a version switcher at the top).

If the documentation doesn't answer it, try the tool's official forums, then a well-asked search.

6.7 Learning with AI assistants without letting them think for you

AI assistants can explain a concept five different ways, spot the typo in a query, and describe what an error message means. Used well, they make you learn faster. Used badly, they let you finish every exercise without learning anything, and you find out in an interview, or in your first week at work, when the assistant isn't the one being asked.

Rules that keep the learning yours

1. **Try first.** Spend real effort on an exercise before asking for help: at least fifteen to twenty minutes, or until you can say exactly where you're stuck. The struggle is where the learning happens.
2. **Ask for explanations, not answers.** "Why does my query return 21 rows instead of 19?" teaches you something. "Write the query for exercise 14" teaches you nothing.
3. **Check everything against a source you trust.** Assistants can be fluent and wrong at the same time. Section 6.6's rounding question is a good test: ask an assistant how 2.5 rounds in Python, then check the documentation and run it yourself.
4. **Never paste data you don't own.** Company data, customer names, invoices, and passwords don't belong in a chat with an outside service unless your employer has explicitly approved that tool for that data (Chapter 2, section 2.9, and Chapter 64 go deeper). Practice data like Riverstone's is fine.
5. **Explain it back.** After the assistant helps, close the chat and explain the idea in your own words, or redo the exercise from a blank page. If you can't, you haven't learned it yet.
6. **Use it as a tutor, not a crutch.** Good requests: "Quiz me on percentage points versus percent change", "Give me a harder version of this exercise", "Explain this error message; don't fix it".

Chapter 26 covers AI assistants at work, and Chapters 54 and 55 explain how they work and how to build with them.

6.8 Building a study habit

The book is long. The difference between finishing and stopping is rarely intelligence; it's a plan, a rhythm, and a way of keeping going after a missed week.

A sample six-month plan

A sample six-month plan at about 8 hours a week

Month 1 Weeks 1-4	Part 0 and Part I: Chapters 1-9 foundations, the map, how learning works
Month 2 Weeks 5-8	Chapters 10-11: spreadsheets Excel and Google Sheets, pivot tables, Power Query
Month 3 Weeks 9-12	Chapters 12-13: SQL queries on the Riverstone database, in PostgreSQL and MySQL
Month 4 Weeks 13-17	Chapters 14-16: cleaning, charts, Power BI a first dashboard
Month 5 Weeks 18-22	Chapters 17-18 and 21-22: Python, statistics pandas, descriptive statistics, not fooling yourself
Month 6 Weeks 23-26	Chapters 19-20 and 23-27: automation, business, portfolio capstone project; start interview practice (Chapters 68-71)

Every week: one review session, the chapter exercises, and a little SQL or Python practice on something you've already learned.

Figure 6.2 — One way through the analyst path (Parts 0 to II) in six months, at about 8 hours a week.

Figure 6.2 is a sample, not a rule. It's ambitious: about 8 hours a week for 26 weeks. At 5 hours a week, stretch it to nine or ten months, and nothing is lost. A few choices behind it:

- **Spreadsheets before SQL, SQL before Python.** Each builds on the last, and together they cover most day-to-day analyst work.
- **Power BI in month 4**, once you can shape data, so the dashboards have something good to show.
- **Automation (Chapters 19-20) after Python**, because Chapter 20 uses it.
- **Interview practice in month 6, not at the end.** Answering questions from Part VIII (Chapters 68-71) while you build your portfolio shows you what to revise.
- **Business analysts** can swap some Python time for Chapters 24 and 25, which cover requirements, stakeholders, and process mapping.

A weekly rhythm

One week, about 8 hours

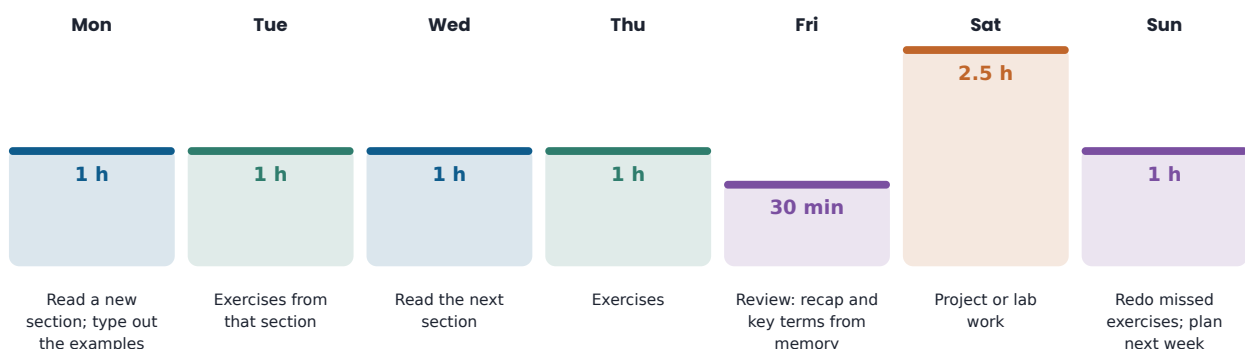


Figure 6.3 — Short, regular sessions, one longer session for projects, and a review built into the week.

Figure 6.3 adds up to 8 hours: $1 + 1 + 1 + 1 + 0.5 + 2.5 + 1$. Three ideas are built into it:

- **Alternate reading and doing.** A section read on Monday gets practiced on Tuesday, while it's fresh but no longer in front of you.
- **Review from memory.** On Friday, close the book and write down the chapter's key terms and main ideas before checking the recap. Pulling ideas out of your memory strengthens them far more than reading them again. Chapter 9 explains why this works and how expertise forms.
- **Plan at the end of the week.** Ten minutes on Sunday deciding exactly which sections you'll do on which days makes Monday simpler to start.

When you fall behind

You will miss weeks. Everyone does. When it happens, don't try to catch up by doubling the hours; that's how a missed week becomes a missed month. Instead: **move the plan back a week, do one short review session to reconnect, and start the next section.** Keep a one-line **study log** in your `notes/` folder (date, what you did, one thing you learned), so that when you return, you can see how far you've come.

6.9 How to use the exercises and answers

Every teaching chapter ends with practice exercises in four groups:

Group	What it's for	How to use it
Warm-up	checks you understood the basics	do them right after reading; they should take a few minutes each
Core	the skills the chapter exists to teach	do all of them; this is where most of the learning happens
Stretch	harder, closer to real work	do at least one; come back to the rest on a second pass
Think about it	judgment, with no single right answer	write a short answer, then compare it with the model answer

The answers are at the end of each chapter in this draft and move to **Appendix G** in the finished book. To get the most from them:

- **Write your answer before you look.** Even a partial answer. Reading an answer you haven't attempted feels like learning and isn't.
- **Compare the reasoning, not only the number.** Many answers show a method or a check (like " $\checkmark$ reconciles to the total"); copy the method.
- **Mark the ones you got wrong or guessed**, and redo them on Sunday from a blank page.
- **For code exercises, run your version.** If it gives the same result by a different route, that's often fine; if it gives a different result, find out why before moving on.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Downloading installers from third-party sites	unwanted extra software; outdated versions	Use each tool's official website or the Microsoft Store
Forgetting the database password	can't connect to PostgreSQL or MySQL	Write it down during installation, somewhere safe
Installing packages without the virtual environment active	"pip installed it, but Python can't find it"	Activate <code>.venv</code> ; use <code>python -m pip install ...</code>
Running commands in the wrong folder	"No such file or directory"	Check where you are (<code>pwd</code>) before running
Trusting a tutorial or AI over the documentation	results that differ between tools, unexplained	Check the official documentation for the version you installed
Letting an AI assistant do the exercises	you can follow solutions but can't write them	Try first; ask for explanations; redo from a blank page
Reading answers before attempting exercises	answers "make sense" but skills don't stick	Write something first, always

In the real world: Meera sets up

Meera Iyer has done analyst work for months without the title: the customer counts in Chapter 1, the January reconciliation in Chapter 3, the board slide in Chapter 4. She wants to become an analyst properly, and she has two computers to do it with. Neither is ideal.

Her work laptop runs Windows 11 and is locked: she can't install software. **Her home laptop** is a MacBook Air from 2020 with 8 GB of memory.

1. Deciding what goes where. She reads section 6.1 and makes a list. The Mac can run everything except Power BI Desktop. The work laptop is where Power BI would be most useful, since the company already uses Microsoft 365. So: the book's main setup goes on the Mac at home; Power BI Desktop goes on the work laptop, if IT agrees.

2. The work laptop. She emails IT with a short, specific request: *"I'd like to install Power BI Desktop from the Microsoft Store for learning and for building sales reports. It doesn't need admin rights. I won't connect it to any company data without approval."* IT approves the same week, and adds that she should use the company's Microsoft 365 account for sign-in when she's ready to share reports.

3. The Mac. She installs Postgres.app, DBeaver, MySQL, Python from python.org (remembering `Install Certificates.command`), VS Code with its Python and Jupyter extensions, and Git with `xcode-select --install`. She creates the `analyst-to-architect` folder, the `.venv`, and runs `check_setup.py`. It reports matplotlib missing: she had typed `matplotlab`. One command later: *All set*. In DBeaver, `SELECT COUNT(*) FROM order_items;` returns 19 in both databases.

4. The 8 GB question. Her Mac slows down with DBeaver, VS Code, twenty browser tabs, and a video call all open. The fix costs nothing: close what she isn't using while studying.

5. The plan. She has about 6 hours a week: an hour on three weekday evenings after work, and three hours on Sunday morning. At that pace, Figure 6.2 becomes a nine-month plan. She writes her first 90 days in `notes/plan.md`: Chapters 7 to 11 by the end of month 2, Chapters 12 and 13 in month 3. At work, she'll rebuild the January reconciliation from Chapter 3 in a spreadsheet as her month 2 practice project, using the practice data rather than company files, so nothing sensitive leaves the company's systems.

6. The AI rule. She sets herself one rule, taped to the edge of her screen: *"Try for 20 minutes. Ask why, not what. Never paste Riverstone's real data."*

She matched tools to computers, asked IT the right way, fixed the one problem the checker found, and wrote a plan for her real week, not an ideal one.

Tools

- **The official websites** for each tool: PostgreSQL, MySQL, DBeaver, python.org, VS Code, Git, and Microsoft for Excel and Power BI. Appendix B lists the current download pages and install steps.
 - `check_setup.py` (companion files, `ch06/`): checks your Python version and the four packages. Tested on Python 3.11.15 and 3.13.13, with pandas 3.0.5, openpyxl 3.1.5, matplotlib 3.11.2, and JupyterLab 4.6.3.
 - **A notes file** (`notes/plan.md` and a study log): a plain text or Markdown file is enough.
-

The project: set up, check, and plan your first 90 days

Goal: a working toolkit, verified, and a study plan you can keep.

Step 1. Install and check. Work through section 6.3. For each tool, record the version and the result of its check:

Tool	Version installed	Check	Result
Spreadsheet		<code>=ROUND(2.5,0)</code>	gives 3
PostgreSQL + DBeaver		<code>SELECT COUNT(*) FROM order_items;</code>	gives 19
MySQL (optional for now)		the same query	gives 19
Python + packages		<code>check_setup.py</code>	prints <i>All set.</i>
VS Code		<code>hello.py</code>	prints <code>ready</code>
Git		<code>git --version</code>	prints a version
Power BI Desktop (Windows)			opens to an empty report

Step 2. Organize. Create the folder structure from section 6.4 and put the companion files in `companion/`.

Step 3. Test the documentation habit. Pick one function you used in Chapter 4 (for example, `MEDIAN` or `RRI`) and find its official documentation page for your spreadsheet. Write down its signature, one example, and one note or warning.

Step 4. Write your 90-day plan in `notes/plan.md` :

- how many hours a week you can study, and on which days and times;
- which chapters you'll finish by day 30, day 60, and day 90 (use Figure 6.2 and adjust for your hours);
- one practice project for each month, using practice data;
- your rule for using AI assistants;
- what you'll do when you miss a week.

Step 5. Start your study log with today's entry.

Deliverable: the completed check table, a screenshot of `check_setup.py` printing *All set.*, and your 90-day plan.

You've got it when...

- ☐ I know whether my computer can run each tool, and I have a plan for any it can't.
- ☐ Every core tool is installed from its official source, and each check passed.
- ☐ `SELECT COUNT(*) FROM order_items;` returns 19 in the Riverstone database.
- ☐ `check_setup.py` prints *All set.*
- ☐ My companion files are in one place, unedited, with my own work in a separate folder.
- ☐ I can open a terminal, see where I am, move into a folder, and run a script.
- ☐ I can find and read the official documentation for a function in each tool.
- ☐ I have rules for using AI assistants that keep the learning mine, and I never paste data I don't own.
- ☐ I have a weekly rhythm and a written 90-day plan.
- ☐ I attempt exercises before reading the answers.

Recap

- **You don't need a high-end computer.** 8 GB of memory is enough to start, 16 GB is comfortable. Power BI Desktop is Windows only; Mac users can run Windows in a virtual machine. Chromebooks cover spreadsheets and online SQL, not Python or Power BI.
- **The core toolkit is free:** a spreadsheet, PostgreSQL and MySQL with DBeaver, Power BI Desktop, Python with VS Code and Jupyter, and Git. Later parts add their own tools.
- **Install in order, from official sources, and check each tool** before moving on. Chapter 12, section 12.3, has the full database steps.

- **Use a virtual environment** for the book's Python packages, and install with `python -m pip install . check_setup.py` confirms your setup.
 - **Keep the companion files unedited** and your own work in a separate folder, with file names that sort and search well.
 - **A few shortcuts and four terminal commands** save hours over the course of the book.
 - **Documentation settles disagreements.** Rounding 2.5 gives 3 or 2 depending on the tool and the number type, and each tool's manual says exactly why.
 - **AI assistants are tutors, not substitutes:** try first, ask why, check against sources, never paste data you don't own, and explain it back.
 - **A plan and a rhythm finish books.** About 8 hours a week covers the analyst path in six months; fewer hours stretch it, and a missed week moves the plan instead of doubling the load.
 - **Attempt every exercise before reading its answer**, and redo the ones you missed.
-

Practice exercises

Warm-up

1. For each computer, say which of the book's core tools it can run, and what the owner should do about the rest: (a) a Windows 11 laptop with 8 GB of memory; (b) a MacBook with 16 GB of memory; (c) a Chromebook; (d) a locked-down work laptop running Windows 11.
2. Match each check to the tool it confirms: (a) `git --version`; (b) `SELECT COUNT(*) FROM order_items`; returning 19; (c) `check_setup.py` printing *All set.*; (d) `=ROUND(2.5,0)` returning 3.
3. A friend's terminal shows `python: can't open file 'check_setup.py': No such file or directory`. List two likely causes and how to check each.
4. Sort these requests to an AI assistant into "helps you learn" or "does the learning for you", and rewrite the ones that don't help: (a) "Solve Chapter 4, exercise 6." (b) "Explain why averaging monthly growth rates overstates growth." (c) "My query returns 21 rows instead of 19. What kinds of mistake cause extra rows?" (d) "Write my 90-day plan."

Core

5. After installing pandas, a script fails with `ModuleNotFoundError: No module named 'pandas'`. Explain the most likely reason, and write the two commands (for your operating system) that would fix it.
6. Using the documentation quotes in section 6.6, predict the result of each, and explain your reasoning: (a) PostgreSQL `SELECT ROUND(4.5)`; (b) PostgreSQL `SELECT ROUND(4.5::double precision)`; (c) Python `round(4.5)` (d) a spreadsheet `=ROUND(4.5,0)` (e) MySQL `SELECT ROUND(45E-1)`;
7. A Python report and an Excel report of the same invoices show totals that differ by ₹1. Using section 6.6, give one possible explanation, and describe how you'd confirm it.

8. You can study 5 hours a week. Adapt Figure 6.3's weekly rhythm to 5 hours, keeping reading, exercises, review, and project work. Then estimate how long the six-month plan in Figure 6.2 would take at that pace.
9. Your company uses a customer list with names, phone numbers, and outstanding balances. You want an AI assistant's help writing a formula that flags overdue customers. Describe how you'd get the help without sharing the real data.

Stretch

- l0. Write a short, specific request to your IT team asking permission to install PostgreSQL, DBeaver, and Python on a work laptop for learning. Include what each tool is for, what data you will and won't connect to, and one question for them.
- l1. Find the official documentation page for Python's `round()` or PostgreSQL's `ROUND` for the version you installed. Write down its signature, one example you ran yourself, and one note or warning. Then find one tutorial or video that explains rounding, and say whether it matches the documentation.
- l2. Draft your 90-day plan using the checklist in the project, with specific chapters for days 30, 60, and 90, and a plan for a missed week.

Think about it (no calculation needed)

- l3. Why does this book recommend free tools, even though many companies use paid ones? What might you still need to learn on the job?
- l4. A friend says, "I'll skip the exercises and read the answers; it's faster." What would you tell them, using section 6.9?
- l5. When is it reasonable to ask an AI assistant for a complete answer, rather than an explanation? Give one example where it's fine and one where it isn't.

Key terms

operating system · RAM (memory) · virtual machine · installer · Microsoft Store · administrator (admin) rights · LTS (long-term support) · PostgreSQL · MySQL · DBeaver · Power BI Desktop · Python install manager · virtual environment · pip · package · VS Code · extension · Jupyter · Git · companion files · terminal · command · path · keyboard shortcut · official documentation · signature · banker's rounding (round half to even) · round half away from zero · floating-point number · AI assistant · study plan · study log · retrieval practice · warm-up, core, stretch exercises

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 7, The Data Landscape**, and **Chapter 8, The Career Tree**, show where the skills you're setting up for lead, and which roles use which tools.
 - **Chapter 9, How Expertise Actually Forms**, explains the learning science behind section 6.8: practice, review from memory, and feedback.
 - **Chapters 10 and 11** use your spreadsheet; **Chapters 12 and 13** use the databases and DBeaver you installed, starting with the Riverstone data you loaded.
 - **Chapter 16** uses Power BI Desktop; **Chapters 17 and 18** use Python, VS Code, and Jupyter, and explain virtual environments properly.
 - **Chapter 26** puts Git to work and covers AI assistants in a professional setting; **Chapter 34** teaches the command line in depth.
 - **Appendix B** keeps current install steps for every tool; **Appendix E** lists all companion files; **Appendix G** holds the answers.
 - **Interview preparation:** Chapter 81 (behavioral, HR, and offer conversations) covers how to talk about how you learned and what you've built, and Chapter 68 explains how data hiring works.
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

- (a) Everything, including Power BI Desktop; 8 GB is enough to start, and closing unused apps helps. (b) Everything except Power BI Desktop; for Chapter 16, run Windows 11 in a virtual machine, or use a Windows computer at work or college. (c) Google Sheets or Excel for the web, and online SQL playgrounds for practice; plan to use a laptop before Chapter 17 for Python, the databases, and Power BI. (d) Nothing until IT approves; ask for Power BI Desktop from the Microsoft Store (no admin rights needed) and for PostgreSQL, DBeaver, and Python, and meanwhile use a home computer or the web tools with practice data.
- (a) Git. (b) The database and DBeaver (with the Riverstone data loaded). (c) Python and the four packages. (d) The spreadsheet.
- (1) The terminal is in a different folder from the script: check with `pwd`, then `cd` into the `ch06` folder. (2) The file name is different, such as `check_setup.py.txt` because extensions are hidden, or the file is still inside a zip: show file extensions (Chapter 2) and unzip the download.
- (a) Does the learning for you. Better: "I got a different answer for Chapter 4, exercise 6. Here's my working; where did my reasoning go wrong?" (b) Helps you learn. (c) Helps you learn. (d) Does the planning for you. Better: "Here's my draft 90-day plan and my available hours. What's unrealistic about it?"
5. pandas was installed into a different Python from the one running the script, usually because the virtual environment wasn't active. On Windows: `.venv\Scripts\activate`, then `python -m pip install pandas`. On macOS or Linux: `source .venv/bin/activate`, then `python -m pip install`

pandas . (Also make sure VS Code's selected interpreter is the one in .venv .)

6. (a) 5 : 4.5 is numeric , and PostgreSQL breaks ties away from zero. (b) 4 : as double precision , the common rule is round half to even, and 4 is even. (c) 4 : Python rounds halves to the even choice. (d) 5 : spreadsheets round halves away from zero. (e) 4 : 45E-1 is an approximate value, so on most systems MySQL rounds to the nearest even number. (All five were run for this chapter, with exactly these results.)

7. The two tools may round halves differently: Python's round() rounds halves to even (and floating-point values like 2.675 are stored slightly below the half), while Excel's ROUND rounds halves away from zero. If each invoice is rounded before the totals are added, small differences can add up to a rupee. To confirm: find invoices whose unrounded values end in exactly .5 of the rounding unit, compare each line's rounded value in both reports, and check whether the differences add up to ₹1. The lasting fix is to agree on one rule and round at the end (Chapter 4).

8. One way: Monday 1 hour reading, Wednesday 1 hour exercises, Friday 30 minutes review from memory, Saturday 2 hours project, Sunday 30 minutes redoing missed exercises and planning: $1 + 1 + 0.5 + 2 + 0.5 = 5$ hours. The six-month plan assumes about 8 hours a week for 26 weeks, roughly 208 hours; at 5 hours a week that's about 42 weeks, or around ten months.

9. Describe the structure, not the data: "I have a table with columns customer_name , due_date , and balance . How do I flag rows where the due date is before today and the balance is above zero?" Or build a small made-up sample (three fake customers with invented numbers) and share that. Test the formula on the fake data, then apply it to the real file on your own computer. If your company has approved an AI tool for internal data, follow its rules instead.

10. One good version: "Hello, I'd like permission to install three free tools on my work laptop for learning data analysis: PostgreSQL (a database, used to practice SQL on sample data), DBeaver Community (an app for writing SQL), and Python from python.org (for data analysis scripts). I'll only use practice datasets from a textbook, and I won't connect them to any company system or customer data without your approval. Could you tell me whether there's an approved way to connect to company data later, if my manager asks for reports?"

11. Answers vary. A complete answer includes the signature (for example, round(number, ndigits=None) in Python), an example the reader ran themselves, and the note about round(2.675, 2) or PostgreSQL's tie-breaking rule. Many tutorials say "Python rounds .5 up", which doesn't match the documentation; noticing that is the point of the exercise.

12. Answers vary. A good plan names specific hours ("Tuesday and Thursday, 8–9 p.m.; Saturday, 9–11 a.m."), specific chapters for each checkpoint, one project a month on practice data, an AI rule, and a concrete plan for a missed week ("move the plan back a week; do one review session; start the next section").

13. Nobody is locked out by cost, and PostgreSQL, MySQL, Python, and Git are widely used at work anyway. The ideas transfer: most SQL runs elsewhere, and Power BI's concepts map to Tableau and Looker (Chapter 16). On the job you may still need to learn a company's warehouse, BI tool, data definitions, and security rules.

14. Reading answers you haven't attempted feels like learning because the answers make sense, but it doesn't build the skill of producing them. The exercises are where most of the learning happens, especially the core group. Suggest a compromise: attempt every warm-up and core exercise, even partially, before reading its answer, and redo the missed ones at the end of the week.

15. It's reasonable when the task isn't the skill you're trying to learn, or when you already understand it and are saving time, and you'll check the result. Fine: asking for a list of keyboard shortcuts for DBeaver, or for a first draft of an IT request that you then edit. Not fine: asking for the answer to a SQL exercise while you're learning SQL, because producing that answer is exactly the skill you need.